Екстремно програмиране
Extreme programming - XP

Увод

В началото на 90-те Кент Бек и Уорд Кънингам са изпитвали по-добри ***начини за разработване на софтуер***, при който всяко нещо да изглежда просто и по-ефективно. Кент обмисля какво прави софтуера **лесен** за създаване и какво го прави **труден**. През март 1996 г. Кент стартира проект в DaimlerChrysler, използвайки нови концепции в разработката на софтуер.

Увод

Това, което Кент осъзнава, е че има четири измерения, чрез които човек може да подобри всеки софтуерен проект. Трябва да подобрите **комуникацията**. Трябва да търсите **простотата**. Трябва да получите **обратна връзка** за това колко добре се справяте. И винаги трябва да продължавате **смело** напред. Комуникация, простота, обратна връзка и смелост са четирите стойности, търсени от XP програмистите.

Увод

XP/Agile SD: Какво е това?

Как да се работи с XP?

- Процеси
- Екипни практики
- Програмиране
- Правила и практики

1: Какво е екстремно програмиране?

Съзнателен и дисциплиниран подход към разработката на софтуер.

- С внимание към **удовлетвореността** на клиентите.
- **Доставка**, когато е необходимо.
- **Реагиране** на променящите се изисквания.

I: Какво е екстремно програмиране?

Екстремното програмиране (XP) всъщност **е съзнателен и дисциплиниран подход** към разработката на софтуер.

Много години, той вече е доказан, например в компании като Bayerische Landesbank, Credit Swiss Life, DaimlerChrysler, First Union National Bank, Ford Motor Company и UBS.

XP е успешен, защото подчертава **удовлетвореността** на клиентите. Методологията е предназначена да доставя софтуера, от който се нуждае вашият клиент, **когато е необходим**.

XP дава възможност на разработчиците да се **адаптират бързо** към променящите се изисквания на клиентите, дори в края на жизнения цикъл.

II: Какво е екстремно програмиране?

Тази методология акцентира върху **работата в екип**. Мениджърите, клиентите и разработчиците са *част от екип*, посветен на предоставянето на **качествен софтуер**. XP прилага прост, но ефективен начин да даде възможност за разработване на стил на групов софтуер. XP *подобрява* софтуерен проект по четири основни начина:

- ***Комуникация.***
- ***Простота.***
- ***Обратна връзка.***
- ***Смелост.***

II: Какво е екстремно програмиране?

XP програмистите *комуникират* със своите клиенти и колеги програмисти. Те поддържат своите дизайни *прости и изчистени*.

Те получават *обратна връзка* след тестват софтуера си от първия ден.

Те доставят системата на клиентите възможно *най-рано* и прилагат промени, както е предложено.

С тази основа XP програмистите са в състояние *смело* да реагират на променящите се изисквания и технологии.

II: Какво е екстремно програмиране?

XP е различно. Много прилича на **пъзел**.

Има много малки парченца.

Отделно парчетата нямат смисъл, но когато се *комбинират* заедно, може да се види пълна картина.

Това е значително отклонение от традиционните методи за разработка на софтуер и води до промяна в начина, по който програмираме.

Промяна в начина, по който програмираме

Софтуерът, който е проектиран да бъде ***прост и елегантен***, не е по-ценен от софтуера, който е сложен и труден за поддръжка. Може ли това наистина да е вярно? **Екстремното програмиране** (**XP**) се основава на идеята, че това всъщност не е вярно. Един типичен проект ще харчи около двадесет пъти повече за хора, отколкото за хардуер. Това означава, че проект, който харчи 2 милиона долара за програмисти годишно, ще харчи около 100 хиляди долара за компютърно оборудване всяка година. Да кажем, че сме умни програмисти и намираме начин да спестим 20% от разходите за хардуер чрез някои много хитри програмни трикове. Това ще направи изходния код по-труден за разбиране и поддържане, но ние спестяваме 20% или 20 хиляди долара годишно, което е голяма икономия.

Промяна в начина, по който програмираме

Ами ако вместо това напишем нашите програми така, че да са ***лесни за разбиране и разширяване***. Можем да очакваме да спестим не по-малко от 10% от разходите за хората. Това ще дойде до 200 хиляди долара, много по-големи спестявания. Това със сигурност е нещо, което вашите клиенти ще забележат.

Друг важен проблем за клиентите са **бъговете**. XP набляга не само на тестване, а на добро тестване. Тестовете са автоматизирани и осигуряват защитна мрежа както за програмисти, така и за клиенти. Тестовете се създават преди да бъде написан кодът, докато кодът е написан и след като кодът е написан.

Промяна в начина, по който програмираме

Тъй като се откриват *грешки*, трябва да се добавят нови тестове. Създава се плътна предпазна мрежа. Бъговете не преминават два пъти и това е нещо, което клиентите ще забележат. Друго нещо, което клиентите ви ще забележат, е отношението на XP програмистите към променящите се изисквания. XP ни позволява да приемем промяната. Твърде често клиентът ще види реална възможност да направи системата полезна, след като бъде **доставена**. XP **съкращава** това, като получава обратна връзка от клиентите по-рано, докато все още има време за промяна на функционалността или подобряване на приемането на потребителите.

Промяна в начина, по който програмираме

Клиентите определено ще забележат това.

Голяма част от това, което влезе в XP, беше преоценка на начина, по който е създаден софтуерът.

Качеството на изходния код е много по-важно, отколкото човек може да си представи. Това, че нашите клиенти не могат да видят нашия изходен код, не означава, че не трябва да полагаме усилия да създадем нещо, с което можем да се гордеем.

Промяна в начина, по който програмираме

Ако обобщим казаното това ще звучи така:

- Поддържайте нещата **прости и елегантни**.

Наблягнете на **тестването**.

- Автоматизирани тестове.
- Създавайте тестове преди, кога и след кодирането.

Когато бъдат открити **грешки (бъгове)**, се добавят нови тестове.

- Прегърнете промяната.
- Поддържайте ранна обратна връзка.

Признайте, че изискванията могат/ще се **променят**.

- постоянна интеграция.

Необходима ли е сега друга методология?

Например, когато телескопът Хъбъл беше изстрелян в космоса, той имаше огледало, изградено според *грешните спецификации*. Едва когато телескопът беше напълно разгърнат и в ръцете на неговите клиенти, проблемът беше *открит*. Клиентите на сателита не могат да опитат да го използват, докато не са в космоса, но защо правим това със софтуер? Ами ако влезете в лаборатория по електроника и видите работни маси, покрити със стари уреди? Бихте ли запазили този огромен стар телевизор с причудливата малка кръгла тръба за картина, защото все още имаше добро захранване? Бихте ли запазили старо AM радио, защото беше полезно като RF генератор?

Необходима ли е сега друга методология?

Не бихте, защото работната ви маса щеше да бъде толкова претрупана, че не можете да свършите нищо. Точно това правим със софтуера.

При друг пример мостът Zilwaukee над река Saginaw трябваше да бъде построен набързо. Мостът е издигнат от двата бряга на реката, за да се събира в средата. Но когато стигнаха до средата, едната страна беше *три фута по-висока от другата*. Когато половината от вашия проект е от едната страна на реката, а другата половина е от другата, не можете да интегрирате проекта си до самия край, но защо правим това със софтуер? Екстремното програмиране (XP) е ***създадено в отговор на този вид въпроси***. XP се основава на наблюдения за това какво прави компютърното програмиране по-бързо и какво го прави по-бавно.

Необходима ли е сега друга методология?

XP е важна нова методология по две причини.

На **първо** място това е преразглеждане на практиките за разработка на софтуер, които са се превърнали в стандартни оперативни процедури.

И **второ**, това е една от няколкото нови *леки софтуерни методологии*, създадени за намаляване на цената на софтуера.

XP отива още една стъпка напред и дефинира процес, който е прост и приятен.

Необходима ли е сега друга методология?

Може да обобщим казаното така:

XP - Въз основа на наблюдения за това какво подобрява/намалява скоростта на програмиране:

- Преразглежда практиките за разработка на софтуер.
- Създава Лека/ Agile (гъвкава) софтуерна методология.
- Намалява цената на софтуера.
- И е просто и приятно за разарботчиците.

Каква е разликата между гъвкавата/лека методология и традиционната?

Традиционна софтуерна методология:

Набор от правила и практики.

тежка категория.

- много правила, практики и документи.

Лека.

- няколко правила и практики или някои, които са лесни за следване.

Каква е разликата между гъвкавата/лека методология и традиционната?

Софтуерната методология е набор от правила и практики, използвани за създаване на компютърни програми. Една тежка методология има **много правила, практики и документи**. Изисква дисциплина и време, за да се следват правилно. Олекотената методология има само **няколко правила и практики** или такива, които са **лесни** за следване. В края на 60-те и началото на 1970-те е обичайна практика компютърните програмисти да създават софтуер по какъвто и да е начин. Много програмисти се отличиха в създаването на софтуер, твърде сложен, за да го разбере някой. По това време беше чудо, ако една програма работи без никакви грешки.

Каква е разликата между гъвкавата/лека методология и традиционната?

Да направиш компютрите полезни се смяташе за достойно търсене и не за разлика от приключението в стария запад. През 1968 г. Edsger Dijkstra написа писмо до CACM, озаглавено GOTO Statement считано вредно. Раждат се централните идеи на ***софтуерното инженерство***. По това време вярвахме, че по-големи, по-дисциплинирани методологии ще ни помогнат да създадем софтуер с постоянно качество и предвидими разходи. Беззаконните «каубойски» кодери бяха овладяни. 80-те години бяха добри времена за компютърните програмисти. Имахме няколко правила и практики за създаване на софтуер, който е много по-добър по качество от това, което създавахме само няколко години по-рано.

Каква е разликата между гъвкавата/лека методология и традиционната?

Изглеждаше, че ако можем просто да създадем достатъчно правила, за да покрием проблемите, които срещаме, бихме могли да създадем перфектен софтуер и да бъдем навреме. Добавяхме все повече и повече правила и практики, за да покрием всички потенциални проблеми. Сега в 21-ви век откриваме, че тези правила са трудни за следване, процедурите са сложни и не са добре разбрани и количеството документация, написана в някаква абстрактна нотация, е извън контрол. Опитът да се измисли по-голяма и по-добра методология беше като златна треска в Калифорния; всички се отправиха на запад само за да бъдат разочаровани.

Agile Software Manifesto

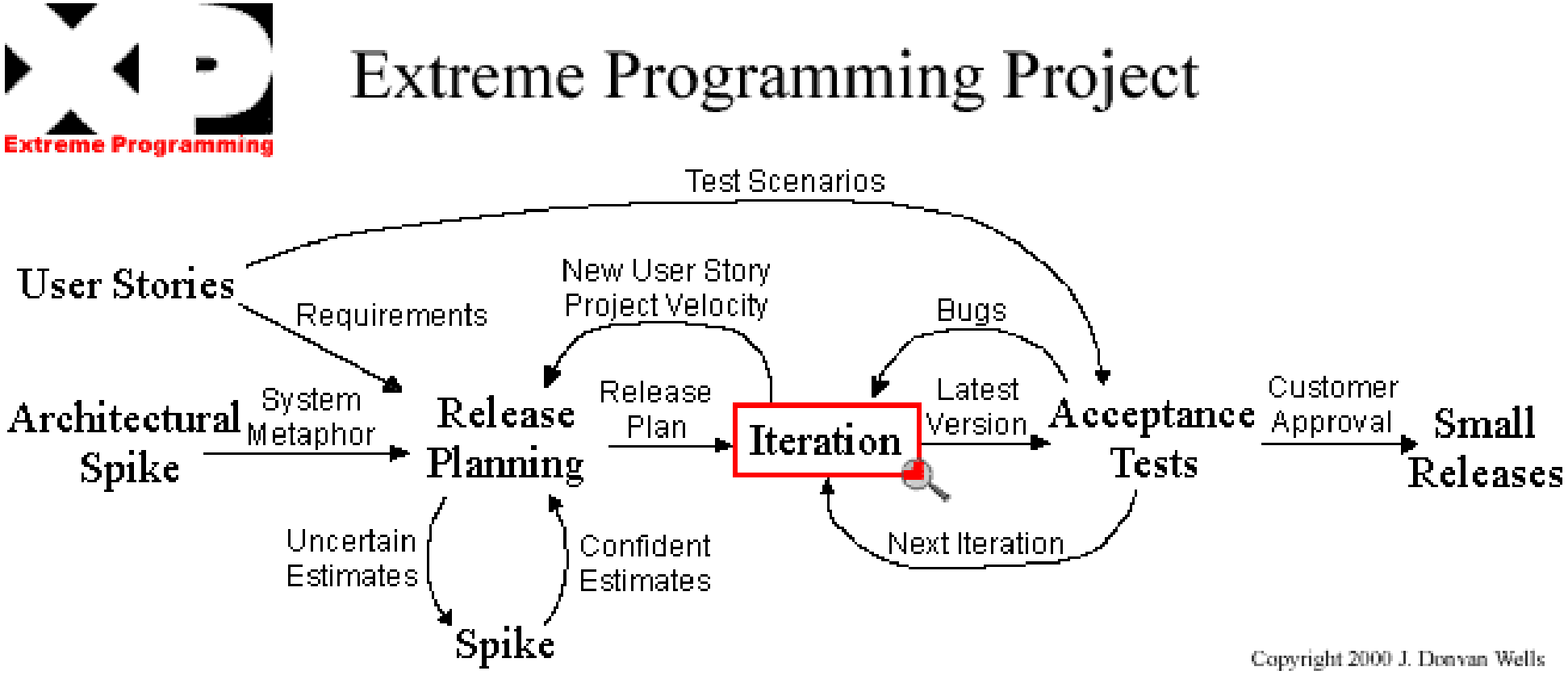
XP, Scrum, DSDM, ASD, Crystal, технологиите се базират на функции за прагматично програмиране (описано в манифеста):

- Така се откриват по-добри начини за разработване на софтуер, който помага на всички да подобрят качеството на тази работа.

При това се оценяват:

- Индивиди и взаимодействия с процесите и инструментите.
- Работещ софтуер върху изчерпателната документация.
- Сътрудничество с клиенти при договаряне на договори.
- Отговаряне на промяната при следване на плана.

Схема на XP


Extreme Programming
Extreme Programming Project
Test Scenarios
User Stories
Requirements
New User Story
Project Velocity
Bugs
Architectural Spike
System Metaphor
Release Planning
Release Plan
Iteration
Latest Version
Acceptance Tests
Customer Approval
Small Releases
Uncertain Estimates
Confident Estimates
Spike
Next Iteration
Copyright 2000 J. Donvan Wells

Потребителски истории (User Stories)

Имат същата **цел** като случаите на употреба (use case - UML).

- Използват се за създаване на прогнози за време за планиране на пускането на версията.
- Заменят документи с големи изисквания.
- Написани са от клиентите.
- Управляват създаването на тестове за приемане.
- Разработчиците създават оценки в „идеалното време за разработка“.
- Набор от избрани потребителски истории и формират план за реализация.
- Фокусирани са върху нуждите на потребителите.

Потребителски истории

Потребителските истории служат за същата цел като случаите на употреба, но не са еднакви.

Те се използват за създаване на **прогнози** за времето за срещата за планиране на реализацията. Те също се използват вместо голям документ с изисквания. Потребителските истории се **пишат от клиентите** като неща, които системата трябва да направи за тях. Те са подобни на сценариите на използване, с изключение на това, че не се ограничават до описание на потребителски интерфейс.

Те са във формат чрез около *три изречения текст*, написан от клиента в терминологията на клиента без техносинтаксис.

Потребителски истории

Потребителските истории също **стимулират създаването на тестовете** за приемане. Трябва да бъдат създадени един или повече автоматизирани тестове за приемане, за да се потвърди, че потребителската история е внедрена правилно. Едно от най-големите недоразумения с потребителските истории е как те се различават от традиционните спецификации на изискванията. Най-голямата разлика е в нивото на детайлност. Потребителските истории трябва да предоставят *достатъчно подробности*, за да се направи оценка на сравнително нисък риск за това колко време ще отнеме историята, за да се приложи. Когато дойде време за внедряване на историята, разработчиците ще отидат при клиента и ще получат подробно описание на изискванията *лице в лице*.

Потребителски истории

Разработчиците оценяват колко време може да отнеме историите за изпълнение. Всяка история ще получи оценка за 1, 2 или 3 седмици в „идеалното време за разработване“. Това идеално време за разработка е колко време би отнело да се внедри историята в код, ако нямаше разсейване, никакви други задачи и знаехте точно какво да правите. По-дълго **от 3 седмици** означава, че трябва да разбиете историята допълнително. **По-малко от 1 седмица** и сте на твърде подробно ниво, комбинирайте няколко истории. Около 80 потребителски истории плюс или минус 20 е идеалното число за **създаване на план** за пускане по време на планирането на пускането.

Потребителски истории

Друга разлика между историите и документа с изисквания е фокусирането върху **нуждите на потребителите**.

Трябва да се опитате да избягвате подробности за конкретна технология, оформление на базата данни и алгоритми.

Трябва да се опитате да държите историите фокусирани върху нуждите и предимствата на потребителите, вместо да указвате оформления на **GUI**.

Потребителски истории - Пример

- История: **Обработвайте овърдрафт**

Когато транзакция кара сметката на клиента да премине в овърдрафт, прехвърлете пари от сметката за защита на овърдрафт, ако има такава.

- История: **Изчисляване на баланса**

За всяка сметка изчислете баланса, като съберете всички депозити и извадете всички удръжки.

Spike архитектура

Спайк решения:

Фигурата отговаря на трудни технически или дизайнерски проблеми.

- Намалете риска от технически проблем.
- Увеличете надеждността на оценката за потребителска история.

Проста програма за изследване на потенциални решения.

- Решавайте само един проблем в даден момент.

Spike архитектура

Създайте спайк решения, за да намерите отговори на трудни технически или дизайнерски проблеми. **Spike решението** е много проста програма за изследване на потенциални решения. Създайте система, която се занимава само с разглеждания проблем и **игнорирайте** всички друго. Повечето спайкове не са достатъчно добри за запазване, така че очаквайте да ги **изхвърлите**. **Целта** е намаляване на риска от технически проблем или повишаване на надеждността на оценката на потребителската история. Когато техническа трудност заплашва да задържи развитието на системата, поставете двойка разработчици върху проблема за седмица или две и намалете потенциалния риск

Изберете системна метафора

Създайте общ речник за системата.

- Последователно ***именуване на класове и методи***.
- Имената трябва да са *лесни за научаване* и *свързване*.

Изберете системна метафора

Изберете системна метафора, за да поддържате в екипа на ***един и същи подход за наименувате*** класове и методи последователно. Това, как ще назовавате вашите обекти, е много важно за разбирането на цялостния дизайн на системата и повторното използване на кода. Да можеш да отгатнеш какво би могло да се нарече нещо, ако вече съществува и да си прав, спестява реално време. Изберете система от имена за вашите обекти, с които всеки може да се свърже, без конкретни, трудно придобивани знания за системата.

Изберете системна метафора

Например системата за заплати на Chrysler е изградена като производствена линия. При Ford продажбите на автомобили бяха структурирани като спецификация на материалите.

Има също така метафора, известна като наивна метафора, която се основава на самия ви домейн.

Но не избирайте наивната метафора, освен ако не е достатъчно проста.

Примери за системна метафора

Наивно

- поточна линия
- Споделена черна дъска
- Подизпълнители
- Работния процес

Редактирана:

Карта - Card(имитация на перфокарта)
Масив от редове Array of lines (vi)
низ (Emacs)
Последователност на изпълнения (Word)
Дърво – Tree (XML)
Комбинации

Среща за планиране на реализацията

- Създайте план за реализацията (изданието).
- Планът за реализацията се използва за създаване на итерационни планове за всяка отделна итерация.
- Има набор от правила за балансиране на технически и бизнес решения/сили за договаряне на график.

Среща за планиране на реализацията

Среща за **планиране на реализацията** се използва за създаване на план за реализацията, който излага цялостния проект. След това планът за реализацията се използва за създаване на **итерационни планове** за всяка отделна итерация. Важно е техническите хора да вземат техническите решения и бизнесмените да вземат бизнес решенията. Планирането на реализацията има набор от правила, които позволяват на всеки, участващ в проекта, да взема свои собствени решения. Правилата определят метод за договаряне на график, към който всеки може да се ангажира. Същността на срещата за планиране на изданието е екипът за разработка да оцени всяка потребителска история по отношение на идеалните седмици за програмиране.

Среща за планиране на реализацията

Идеалната седмица е колко време си представяте, че ще отнеме да приложите тази история, ако нямате абсолютно нищо друго да правите. Без зависимости, без допълнителна работа, но включва тестове. След това клиентът решава **коя история е най-важна** или има най-висок приоритет за завършване. Потребителските истории са отпечатани или написани на **карти**. Заедно разработчиците и клиентите преместват картите на голяма маса, за да създадат набор от истории, които да бъдат приложени като първа (или следваща) версия. Желателна да се използва тестируема система, която прави добър бизнес смисъл, доставян рано. Можете да планирате по време или по обхват.

Среща за планиране на реализацията

Можете да **планирате по време или по обхват**. Скоростта на проекта се използва, за да се определи или колко истории могат да бъдат изпълнени преди дадена дата (час) или колко време ще отнеме завършване на набор от истории (обхват). Когато планирате по време, умножете броя на повторенията по скоростта на проекта, за да определите колко потребителски истории могат да бъдат завършени.

Когато **планирате по обхват**, разделете общите седмици на очакваните потребителски истории на скоростта на проекта, за да определите **колко итерации до готовността на изданието**.

Среща за планиране на реализацията

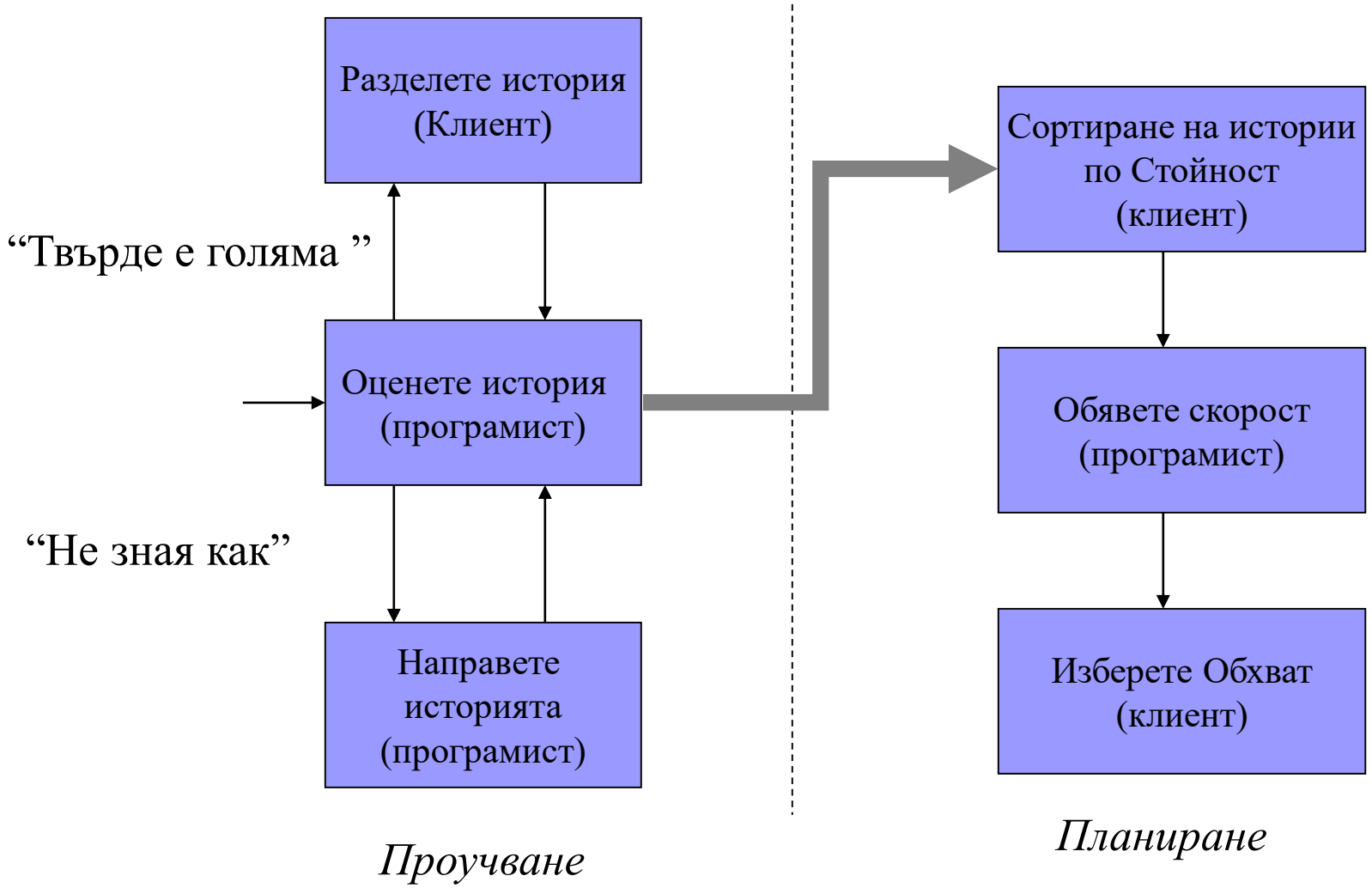
- Планът за реализацията се основава на оценки на ***разработчиците*** за набор от потребителски истории и ***клиентски приоритети*** (време/обхват).
- ***Клиентът*** решава приоритетите на потребителските истории. Най-важното е ***първо***.
- ***Потребителските истории*** се избират за итерация въз основа на време или обхват. Желателна е ранна доставка с бизнес стойност.

Планът за реализацията

Съдържа:

- ***потребителски истории***, които да бъдат внедрени за всяка версия на системата.
- ***дати*** за тези ***издания***.

Планиране на реализацията



I: Приемен Тест

- Създаден от потребителски истории.
- Итерация: избраните ***потребителски истории*** се превеждат в тестове за приемане.
- Сценарии: определени от клиента за тестване за ***правилно изпълнение*** на потребителска история.
- Една история може да има един или много тестове за приемане.
- Това е черна кутия за системни тестове.
- **Клиентът** проверява *правилността* и преглежда резултатите от тестовете, за да реши *приоритета* на неуспешните тестове.
- Използва се като регресионни тестове преди пускането в производство.

I. Приемен Тест

Тестовете за приемане се създават от **потребителски истории**. По време на итерация потребителските истории, избрани по време на срещата за планиране на итерацията, ще бъдат преведени в тестове за приемане. Клиентът посочва сценарии за тестване, когато дадена потребителска история е била правилно внедрена. Една история може да има един или *много тестове* за приемане, каквото и да е необходимо, за да се гарантира, че функционалността работи. Тестовете за приемане са системни тестове с черна кутия. Всеки тест за приемане представлява някакъв очакван резултат от системата. Клиентите са *отговорни* за проверка на *коректността* на тестовете за приемане и преглед на резултатите от тестовете, за да решат кои неуспешни тестове са с най-висок *приоритет*.

I. Приемен Тест

Тестовете за приемане се използват и като *регресионни* тестове преди пускането в производство. Потребителска история не се счита за завършена, докато не премине своите тестове за приемане. Това означава, че нови тестове за приемане трябва да се създават всяка итерация или екипът за разработка ще отчете нулев напредък. ***Осигуряването на качеството*** (QA) е съществена част от процеса на XP. При някои проекти QA се извършва от отделна група, докато при други QA ще бъде интегриран в самия екип за разработка. И в двата случая XP изисква разработката да има много по-тясна връзка с QA.

I. Приемен Тест

Тестовете за приемане трябва да бъдат ***автоматизирани***, за да могат да се изпълняват често. Резултатът от теста за приемане се публикува на екипа. Отговорност на екипа е да планира времето за всяка итерация, за да коригира всички неуспешни тестове. Тестовете за приемане на името бяха променени от функционални тестове. Това по-добре отразява намерението, което е да се гарантира, че изискванията на клиента са изпълнени и системата е приемлива.

II. Приемен Тест

- Потребителската история не е завършена, ***преди успешното и приемане след тест*.**

  - Нови тестове за всяка итерация или разработката ще отчита нулев напредък.

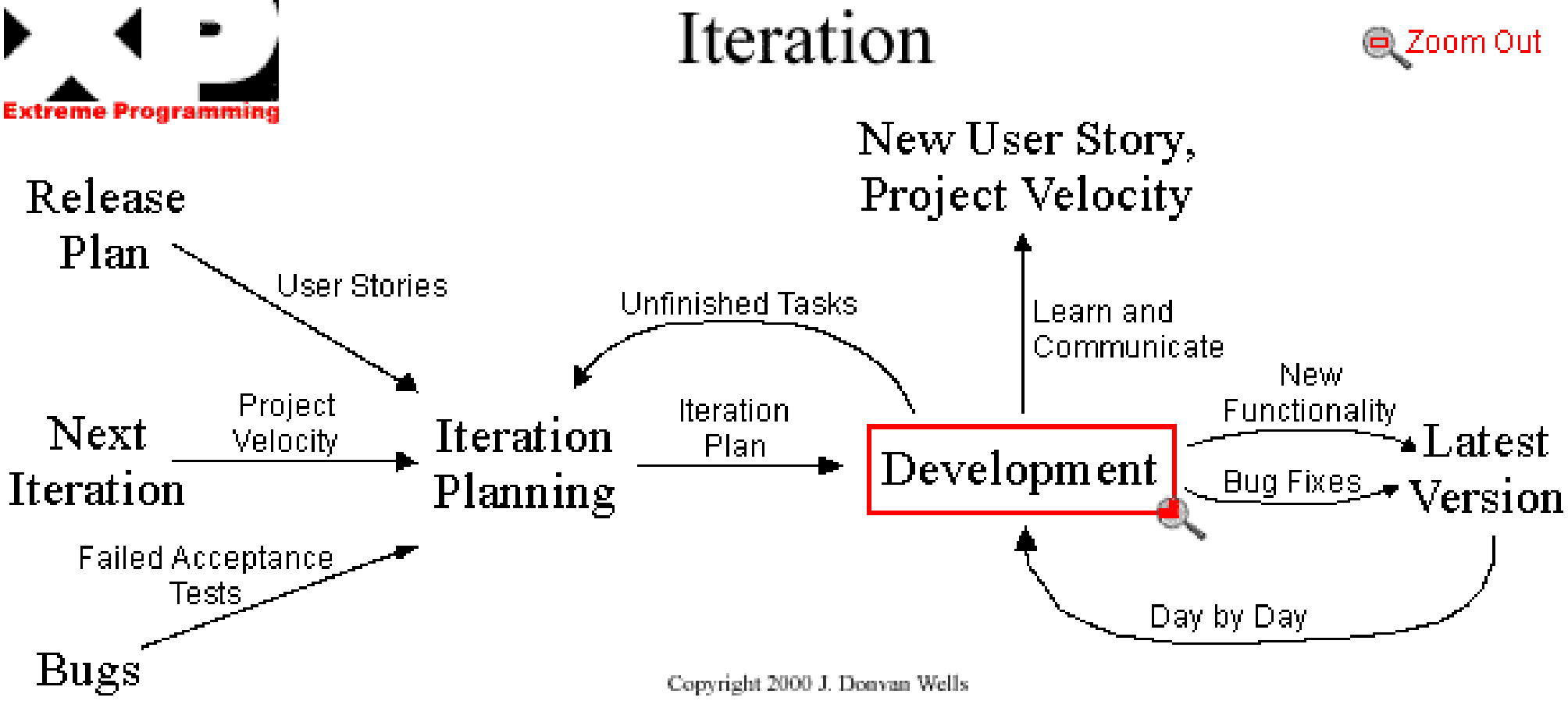
- Трябва да бъде ***автоматизиран***, за да се изпълнява често.
- Екипът на XP планира време, за да коригира всички неуспешни тестове за всяка итерация.

Малки издания (реализации)

- Препоръката е: Пускайте често итеративни версии на клиентите.
- Среща за планиране на освобождаване, за да откриете малки функционални единици, които:
  - Имат добър *бизнес смисъл*
  - Реализирайте *удобната среда* за клиента още в началото на проекта.
- От решаващо значение за получаването на ценна обратна връзка *навременатa* реализаци.
- Колкото по-дълго чакате да представите важна функция, толкова по-малко време ще имате да я *поправите*.

Итерации

Extreme Programming


Iteration
Zoom Out
New User Story,
Project Velocity
Release
Plan
User Stories
Unfinished Tasks
Learn and
Communicate
New
Functionality
Next
Iteration
Project
Velocity
Iteration
Planning
Iteration
Plan
Development
Latest
Version
Bug Fixes
Failed Acceptance
Tests
Day by Day
Bugs
Copyright 2000 J. Donvan Wells

Итеративна разработка

- ***Разделете*** графика на около дузина ***повторения*** с продължителност от **1 до 3** седмици.
- Не планирайте задачите за програмиране предварително.
- Използвайте ***срещи*** за планиране на итерации преди всяка итерация.
- Не внедрявайте нищо предварително (не е планирано за тази итерация).
- По-лесно е да се справите с променящите се изисквания, когато:
  - никога не добавяйте функционалност рано.
  - практикувайте планиране точно навреме.

Итеративна разработка

Итеративната разработка добавя ***гъвкавост*** към процеса.

Вместо това имайте среща за планиране на итерацията в началото на всяка итерация, за да планирате какво ще бъде направено. Също така е против правилата да гледате напред и да се опитвате да приложите всичко, което не е предвидено за тази итерация. Ще има достатъчно време за прилагане на тази функционалност, когато тя стане най-важната история в плана за пускане. Когато никога не добавяте функционалност рано и практикувате планиране ***точно навреме***, е лесно да останете в крак с променящите се потребителски изисквания.

Планиране на скоростта

- Мярка за това колко бързо се извършва работата.
  - Брой потребителски истории/задачи за програмиране, завършени по време на итерация.
  - Сравнете с оценките.
- Използва се като вход за оценка колко нови истории да бъдат направени в следващата итерация.

Планиране на скоростта

Скоростта на проекта (или просто скоростта) е мярка за това ***колко бързо се извършва работата по вашия проект***. Доскоро за тази цел се използваше *коефициентът на натоварване*. Изглежда, че скоростта на проекта е по-лесно за използване. Ако това помогне, използвайте фактора на натоварване, за да създадете първоначална оценка на скоростта на проекта, но след това измерете скоростта и го използвайте вместо това. За да ***измерите скоростта*** на проекта, просто преброявате колко потребителски истории или колко задачи за програмиране са били завършени по време на итерацията. Сумирайте оценките, които са получили тези истории или задачи.

Планиране на скоростта

По време на ***срещата за планиране*** за одобряване на реализацията скоростта на проекта в завършените истории може да се използва, за да се прецени колко истории ще бъдат изпълнени. По време на планирането на итерацията на разработчиците е разрешено да се регистрират, за да изпълняват същия брой прогнозни дни задачи по програмиране, равен на скоростта на проекта, измерена в предишната итерация. Този прост механизъм позволява на разработчиците да се възстановят и почистят след трудна итерация. Скоростта на вашия проект се *увеличава*, като позволява на разработчиците да поискат от клиентите друга история, когато работата им приключи по-рано и не остават задачи за изпълнение.

Планиране на скоростта

Не се притеснявайте да разделяте скоростта на проекта на дължината на итерацията или броя на разработчиците. Това число не е добро за сравняване на производителността на два проекта, тъй като всеки проектен екип ще има различно пристрастие към оценката на историите и задачите, някои оценяват високо, някои оценяват ниски. Проследяването на общото *количество извършена работа* по време на всяка итерация е ключът към поддържането на проекта на равномерено темпо. Трябва да използвате среща за планиране на пускане на реализацията, за да преоцените и ***предоговаряте*** плана, ако скоростта на вашия проект се промени *драстично* за повече от една итерация. Очаквайте скоростта на проекта да се промени, когато системата бъде пусната поради изпълнение на различни задачи по поддръжката.

Благодаря за вниманието!

УСП - 3 НН